MLP Function Approximation

Rapport de Projet

Abdallah Mohammed Ramzi

1. Introduction

Ce rapport présente un projet d'approximation de fonction non linéaire par un Perceptron Multicouche (MLP) implémenté depuis zéro avec NumPy. L'objectif est d'apprendre le mapping $(x, y) \to z = f(x, y)$, une fonction oscillatoire bidimensionnelle sur $[-5, 5]^2$, à partir de 2 000 points générés aléatoirement.

$$f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right) + 0.5\,\cos(2x + 2y)$$

2. Architecture du Réseau

Le réseau adopte une structure $[2 \to 128 \to 128 \to 64 \to 1]$: deux entrées, trois couches cachées avec activation **Tanh**, et une sortie linéaire pour la régression. Les poids sont initialisés par la méthode **He**.

Couche	Neurones	Activation	Rôle
Entrée	2	—	Coordonnées $(x,\, y)$
Cachée 1	128	Tanh	Capture des non-linéarités
Cachée 2	128	Tanh	Représentations profondes
Cachée 3	64	Tanh	Compression des features
Sortie	1	Linéaire	Valeur réelle z

3. Méthodologie et Hyperparamètres

3.1 Split Train/Test

Le dataset de 2 000 points a été divisé en deux ensembles :

- **Entraînement :** 1 600 points (80%) utilisés pour l'optimisation
- **Test :** 400 points (20%) utilisés pour évaluer la généralisation

La normalisation Z-Score a été calculée uniquement sur l'ensemble d'entraînement, puis appliquée au test set pour éviter la fuite d'information.

3.2 Learning Rate — $\eta = 0{,}15$

Après essais comparatifs, la valeur $\eta = 0{,}15$ a donné la meilleure convergence globale avec l'optimiseur *SGD + Momentum*. Cette valeur est plus agressive, ce qui peut produire quelques pics transitoires de loss, mais elle accélère fortement la descente et améliore la performance finale.

3.3 Momentum — $\beta = 0{,}9$

Un terme de momentum a été ajouté dans la mise à jour des poids pour accumuler une moyenne des gradients passés. Cela permet :

- d'accélérer la convergence dans les directions cohérentes,
- de réduire les oscillations locales,
- d'obtenir une descente plus stable en fin d'entraînement.

3.4 Taille de batch — Mini-Batch

L'entraînement utilise des mini-batchs de 32 exemples. Ce choix est justifié par :

- Les mises à jour sont plus fréquentes, ce qui accélère la convergence
- Le calcul reste vectorisé avec NumPy à l'échelle de chaque mini-batch
- Le bruit léger des mini-batchs aide à sortir de certains plateaux

Hyperparamètre	Valeur	Justification
Learning Rate (η)	0,15	Convergence rapide avec mini-batchs
Batch (train)	Mini-batch (32)	Mises à jour plus fréquentes
Époques	1 000	Convergence très rapide observée
Optimiseur	SGD + Momentum	Accélération de la convergence
Momentum (β)	0,9	Lissage des mises à jour, moins d'oscillations
Loss	MSE	Standard pour la régression
Split	80/20	1 600 train / 400 test
Normalisation	Z-Score (train only)	Calculée sur train, appliquée à test

Les formules clés de l'entraînement :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \qquad \frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{2}{N}(\hat{y} - y) \tag{1}$$

$$v_t = \beta v_{t-1} + (1-\beta)\nabla W_t \tag{2}$$

$$W \leftarrow W - \eta\, v_t, \qquad b \leftarrow b - \eta\, v_t \tag{3}$$

4. Résultats et Analyse des Performances

4.1 Performance numérique

Après 1 000 époques d'entraînement (avec $\eta = 0{,}15$, $\beta = 0{,}9$ et des mini-batchs de 32) :

— **MSE sur l'ensemble d'entraînement :** 0.00016

— **MSE sur l'ensemble de test :** 0.00022

L'écart entre train et test reste très faible, ce qui indique une **très bonne généralisation** et l'absence de surapprentissage significatif.

On observe quelques **pics transitoires** de loss durant l'entraînement, mais la tendance globale reste fortement décroissante et la convergence finale est excellente.

4.2 Visualisation

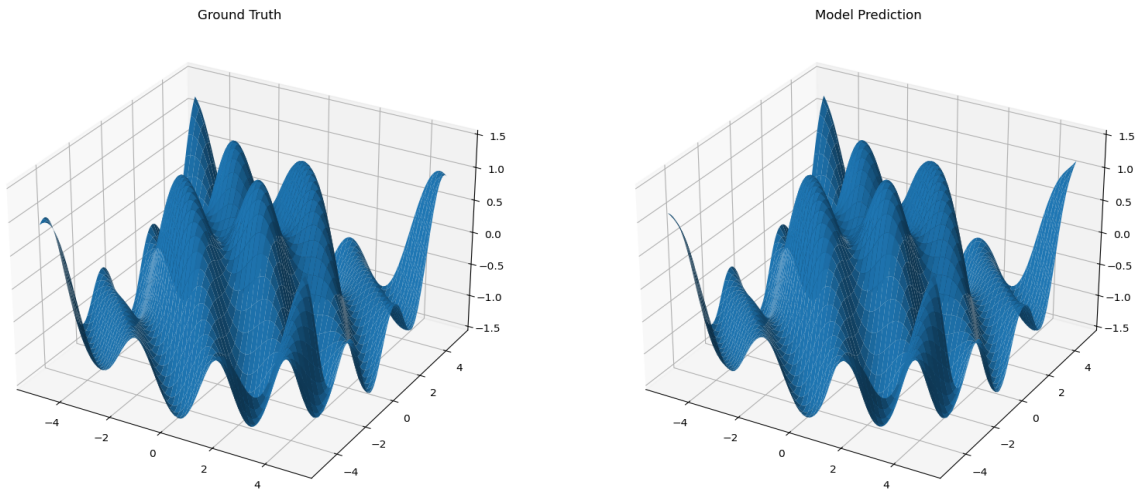

Figure 1 – Gauche : vérité terrain $f(x, y)$. Droite : prédiction du MLP après 1 000 époques avec mini-batchs.

La comparaison des deux surfaces (générées sur une grille régulière de 100×100 points) montre que le MLP a bien appris la **structure globale** de la fonction : les oscillations, les pics et les creux sont fidèlement reproduits. On observe de légères différences dans les

zones périphériques, notamment aux bords de l'espace $[-5, 5]^2$ où les données sont moins denses. Ce comportement est attendu pour un modèle de cette capacité.

Le fait que le modèle généralise bien sur la grille (données non vues) tout en maintenant une faible erreur de test confirme que l'architecture choisie et la procédure d'entraînement sont bien adaptées à l'approximation de fonctions non linéaires oscillatoires.

5. Conclusion

Le MLP $[2 \to 128 \to 128 \to 64 \to 1]$ avec activations Tanh, entraîné par SGD avec Momentum ($\eta = 0{,}15$, $\beta = 0{,}9$) et mini-batchs de 32 sur 1 000 époques, approxime avec succès la fonction cible.

Les résultats démontrent :

— Une excellente **convergence** : MSE d'entraînement d'environ 1.6×10^{-4} en 1 000 époques

— Une excellente **généralisation** : MSE de test d'environ 2.2×10^{-4} sur des données non vues

— Une bonne **approximation visuelle** : les deux surfaces prédites et vérité terrain sont très similaires

— Une robustesse **sur données nouvelles** : la grille régulière (100×100 points non vus) montre une très bonne généralisation

Ce succès est attribué à :

— La normalisation Z-Score appliquée correctement (statistiques calculées sur train uniquement, puis appliquées au test)

— La profondeur du réseau (3 couches cachées avec 128, 128, 64 neurones) pour capturer les oscillations complexes

— L'activation Tanh bien adaptée aux fonctions oscillatoires continues (par rapport à ReLU)

— Le couple $\eta = 0{,}15$ et $\beta = 0{,}9$ combiné aux mini-batchs de 32, qui accélère la convergence tout en gardant un bon niveau de stabilité

— La validation par split train/test (80/20) qui quantifie rigoureusement la généralisation

En résumé, ce projet démontre qu'un MLP implémenté *from scratch* avec NumPy peut avec succès approximer des fonctions non linéaires oscillatoires complexes, sous réserve d'une architecture bien choisie, d'une normalisation correcte et d'une procédure de validation rigoureuse.

Projet réalisé par Abdallah Mohammed Ramzi — MLP from scratch with NumPy